

MATLAB TUTORIAL #1

getting started

This tutorial is broken up into sections. The first section shows how to get started with matlab for this class and how to operate the simple environment that has been created for this class. The second section shows how matlab works with data and variables. The third section discusses the matlab environment and how to operate in it. The last section shows how to plot data and how to work with matlab script/function files and directories.

Please feel free to explore the matlab GUI and the various commands you find. For each command below, observe what is changing in the matlab GUI as you press enter for each command. Trial and error is a good way to explore this language and learn what it can do. As this semester progresses along you will get more and more comfortable using matlab. I think you will find it very powerful and fairly easy to use.

A. Getting Started with Matlab

1. Log into your computer that has matlab installed
2. Create a new directory, perhaps on the desktop, called **ECE4226**, or other suitable name
3. Login to the website, download the file **matlab.zip** and put it in the **ECE4226** directory
4. Open windows explorer, change directory to the **ECE4226** directory, right click on the **matlab.zip** file and click *extract*. This will create a series of directories and files on the hard drive.
5. start matlab and change directory to the **ECE4226/matlab** directory

6. in the command window, type

```
>> init
```

this will modify the matlab path so special class functions are always available to the user.

7. create a new directory for the current project, for example to create a directory for **project1** type

```
>> mkdir project1
```

8. change directory to the lab you are currently working on

```
>> cd project1
```

now you are ready to develop code for this project

9. Matlab continuation on another day. When you want to work on **project1** another day, do the following

start matlab

change directory to the **matlab** root

in the command window, type

```
>> init
```

change directory to **project1** by typing

```
>> cd project1
```

B. Working with Data and Variables

1. Define a vector of zeros or ones

```
>> a = zeros(1,4)
```

this is a *row* vector.

```
>> b = ones(3,1)
```

this is a *column* vector.

Notice that the variable is displayed on the screen in a visual format consistent with the vector shape.

2. Define a vector with uniformly-spaced values

```
>> a = 0:0.1:2
```

The colon operator can be used to define a vector where each component is determined by a fixed numeric step. In the example above, the form of the vector definition is

```
first:step:last
```

so the first value is 0, the step is 0.1, and the upper limit on the numbers is 2. So the vector values are 0, 0.1, 0.2, 0.3, ... up to 2. The **last** value may not actually appear in this definition unless an integer **k** can be found where **last = first + k*step**.

3. Define a vector with arbitrary values

```
>> c = [6 5 4]
```

This example is a *vector* where the values are defined directly.

4. Use the *semi-colon* to suppress a statement output

```
>> a = [0:0.1:2];
```

The variable **a** is not displayed but its values are set by the statement.

5. Display the contents of a variable

```
>> a
>> disp(a)
>> disp(['a = ' num2str(a)])
```

These are 3 different ways to display the vector **a**. The **disp** matlab command stands displays the vector that is enclosed in parenthesis. The vector inside can be numeric, as in the 2nd line above, or all characters as in the 3rd line above. The **num2str** matlab function converts the vector **a** into a string formed by the values in vector **a**.

6. Use **fprintf** to print the contents of a variable

```
>> b = 5;
>> fprintf("b = %5.2f \n",b);
```

where **%5.2f** is a format specifier. The **\n** characters at the end of required to provide a carriage return.

This function is commonly used in programing languages and is universal in its usage.

7. Access one value in a vector using an *index*

```
>> a(2)
```

The integer inside the vector is called an *index* and must be an integer. If a vector has 21 components, then the vector values can be accessed by using an integer index ranging from 1 to 21.

All matlab vectors are *unity-based* indexed which means the first value in the vector is accessed by index 1.

8. Allocate a vector dynamically by appending onto the end (this method is discouraged)

```
>> y = [];           % Default y to the empty vector
>> y = [y 5];        % Append the component 5 to vector y
>> y = [y 3];        % Append the component 3 to vector y
```

The last line above says that vector **y** is constructed as a vector with the first element being the row vector **y** and the last element being the number 3. In this way, a vector can be extended in length.

9. Retrieve the last computational answer. The last answer is always stored in variable **ans**

```
>> ans
```

10. Perform element-wise multiplication/division/exponentiation, use operations: **.*** , **./** , **.^**

```
>> a = [7 3]; b = [2 4]; c = a .* b
```

For these cases, note that the multiplication operation is preceded by a period. The period means to do the operation *component by component*. For this to work, the dimensions of **a** and **b** must be the same.

```
>> d = a.^2
```

Each element of the vector **a** is squared.

```
>> e = a./b
```

Element by element division.

11. Find the *smallest* element of a vector using the matlab intrinsic function *min*

```
>> emin = min(e);
```

12. Find the *largest* element of a vector using the matlab intrinsic function *max*

```
>> emax = max(e);
```

C. The Matlab Environment

1. List all the current variables

```
>> whos
```

2. Clear the variable **a**

```
>> clear a
>> whos
```

notice that the variable **a** is no longer defined.

3. Clear all the variables

```
>> clear all
```

4. Clear the MATLAB window screen

```
>> clc
```

5. See the current MATLAB working directory

```
>> pwd
```

6. Look at the files in the current working directory

```
>> dir
```

D. Working with Signals and Plotting

The matlab signal requires defining two vectors, such as $\{t, x\}$, that define the different points of the signal over time. The time vector t is typically composed of uniformly spaced time samples, where the time spacing is called the *sample time* of the signal. The two vectors t and x must be the same length to define a proper signal so that each time value from vector t corresponds to a unique sample value from vector x .

1. Define and plot a sinusoidal signal with a frequency of 2 Hz using the matlab intrinsic function *sin*

```
>> T = 0.01;           % define the sample time
>> t = -2:T:2;         % define the time vector
>> f0 = 2;
>> x = sin(2*pi*f0*t);  % construct the signal
>> plot(t,x);
>> plot(t,x,'-.r');     % plot the signal and observe the sample points
```

2. Get *help* on a matlab intrinsic function

```
>> help plot
```

3. Making multiple plots in the same figure

```
>> hold on;
>> plot(t,0.5*x,'-.r');
>> hold off;
```

4. Annotating a plot

```
>> xlabel('time, sec');
>> ylabel('sin')
```

5. Making a string vector that includes variable values

```
>> tt = ['sin(2*pi*' num2str(f0) '*t)']
>> title(tt)
```

6. Type **help** on the following *matlab* plotting functions:

```
stem , semilogx , semilogy , loglog
```

E. Working with Matlab Files and Directories

1. Sequences of MATLAB commands can be stored in a file, called a *script*, and executed
2. Create a subdirectory, move to the new directory, and create a script called **myscript.m**

```
>> mkdir test
>> cd test
>> edit myscript.m
```

3. Put some matlab code in the script file

```
clear all; close all; clc
b = 1+1
c = b + 3;
d = b*c
```

4. Save the file, go to the MATLAB command window, and execute the script file

```
>> myscript
```

or

```
>> run('myscript')
```

the command window will show the following result:

```
a = 1 2 3
hello
```

5. Use a looping construct in the script file by replacing the file with the following lines

```
for kk = 1:0.1:1.5
    fprintf('current index, kk = %3.1f \n',kk);
end
```

execute this code by going to the *matlab* command window and typing the filename. The output will be:

```
current index, kk = 1.0
current index, kk = 1.1
current index, kk = 1.2
current index, kk = 1.3
current index, kk = 1.4
current index, kk = 1.5
```

6. Use the `mkdir` command, to create multiple directories as needed in order to *organize* your code.

All your directories will be put in the *matlab* path when you restart matlab as described in the Section I above.

7. The matlab function. Create and edit the matlab file `myfnc.m`. Put the following lines of text in this file.

```
% header for function myfnc
function c = myfnc(a,b)
    c = (a+1)*(b-1);
end
```

Now execute the following lines of code in the MATLAB command window

```
>> x = 3;
>> y = 2;
>> z = myfnc(x,y);
>> z
```

and the correct result can be seen.

Note that any line which begins with the character % is a comment line. Executing the `help` command always shows the comment lines in the matlab function file up to the first non-comment line.

```
>> help myfnc
```

It is also possible to cause a syntax error when executing a function

```
>> z = myfnc(x);
```

which will cause an error to occur.

Not enough input arguments.

Error in myfnc (line 3)
c = (a+1)*(b-1);

the underscores indicate hyperlinks. You can go to the function by clicking on `myfnc` or you can go to line 3 of that function by clicking `line 3`.